SESSION 2
NODE.JS ESSENTIALS

Learning Objectives

In this session, students will learn to:

- Describe the architecture of Node.js
- List benefits and limitations of Node.js applications
- Explain the concept of console-based applications in Node.js
- Explain major programming constructs in Node.js
- Compare the roles of Node Package Manager (NPM) and Node Version Manager (NVM) in Node.js
- Describe global objects and their use in Node.js applications

Node.js is a server-side environment that uses JavaScript to perform request-response operations. It has its own set of benefits and limitations in the computing domain. Interactive and non-interactive command-line environments are popular in Node.js. Programming essentials such as variables, multiline expressions, data types, functions, and arrays are part of Node.js application development. In Node.js, Node Package Manager (NPM) and Node Version Manager (NVM) provide information on packages and versions, respectively.

This session will cover the architecture of Node.js in detail, including the components and workflow. It will explore the benefits and limitations of Node.js applications. The session will describe the concept of a console-based

application and Read Print Evaluate Loop (REPL) environment. It will also explain the important programming constructs, such as data types, functions, and arrays, used in Node.js applications. Finally, the session will conclude by comparing the roles of NPM and NVM in Node.js and using global objects in Node.js applications.

2.1 Introduction to Node.js Architecture

Node.js is an efficient runtime environment based on JavaScript. It is a single-threaded platform in which a single main thread handles all the requests from the clients. Node.js was developed on Google Chrome's V8 JavaScript engine.

Figure 2.1 depicts the visual representation of the Node.js architecture.

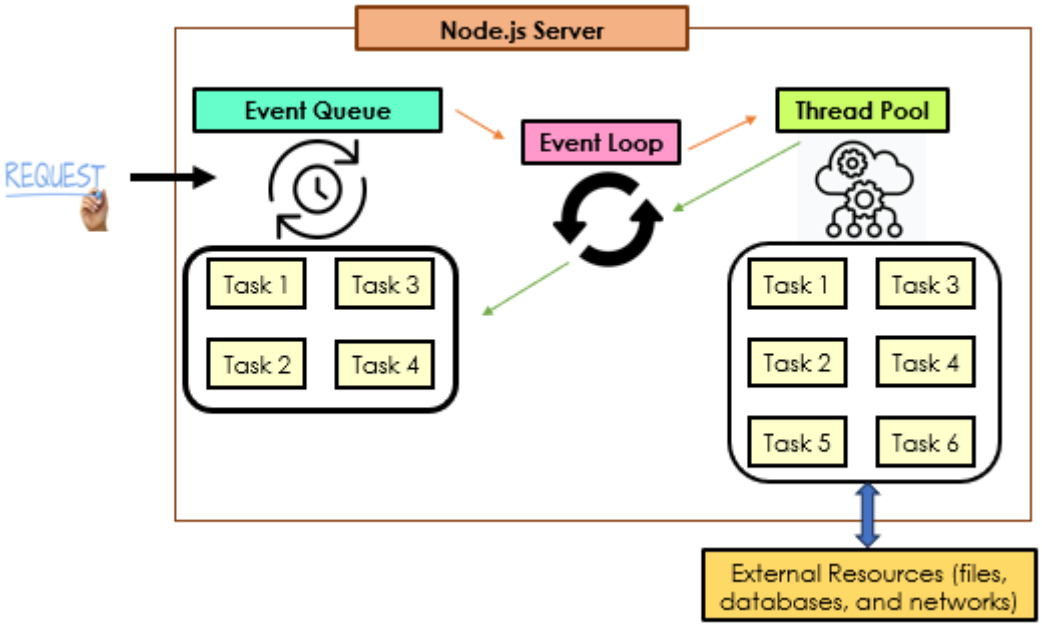
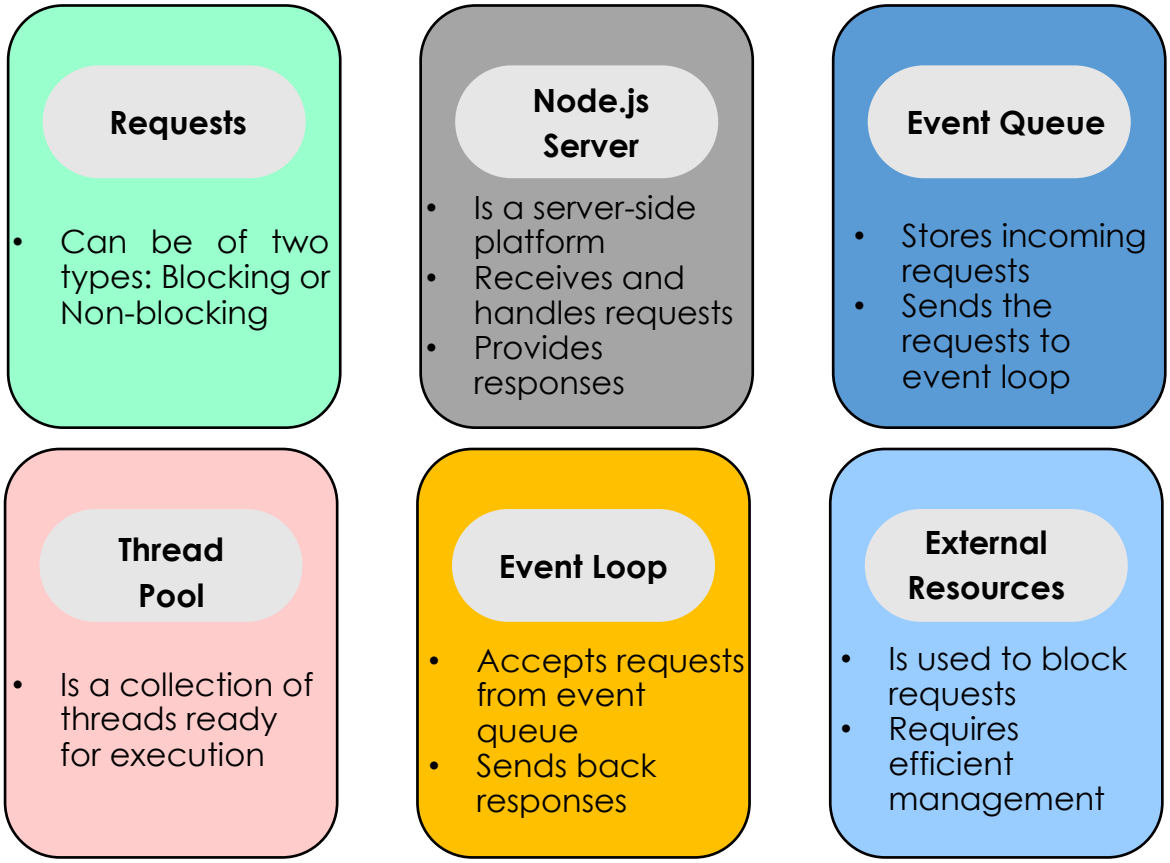


Figure 2.1: Node.js Architecture

2.1.1 Components of Node.js Architecture

Components included in the Node.js architecture are as follows:

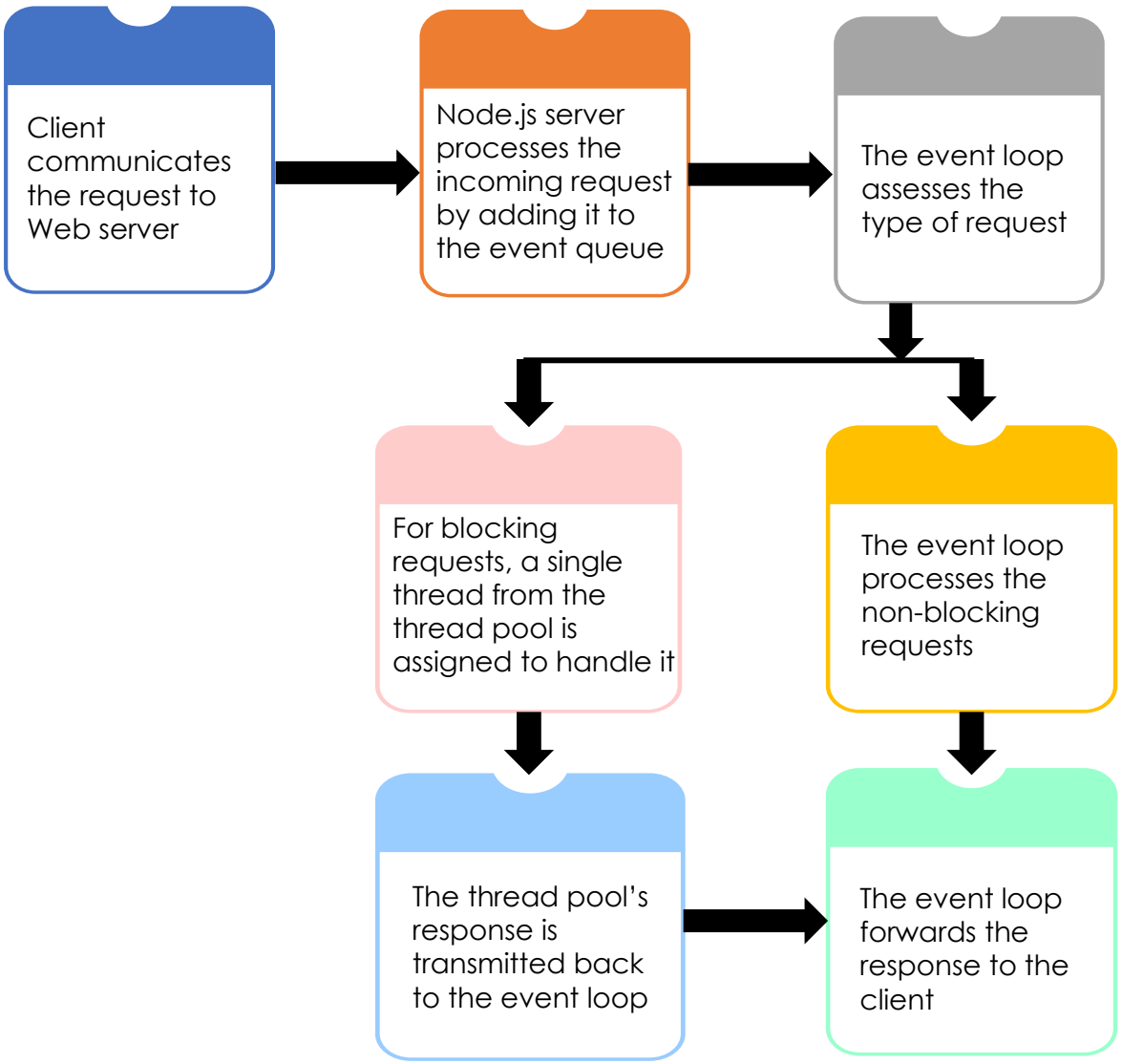


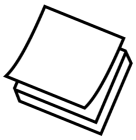
2.1.2 Workflow of Node.js

Node.js follows the concurrent mode of executing tasks. This means that it does not wait for the current task to end and then, begin the execution of the next task. There is a wastage of CPU power if Node.js waits for each task to end before proceeding with the next task.

When the client sends a request to the server, a thread is assigned to that request. The thread is responsible for processing the request from start to end. The thread contains specific instructions to process the request and generate a response. The response generated by the thread is sent to the event loop. The event loop is in charge of transmitting the response back to the client.

Here is the visual representation of the workflow of Node.js.



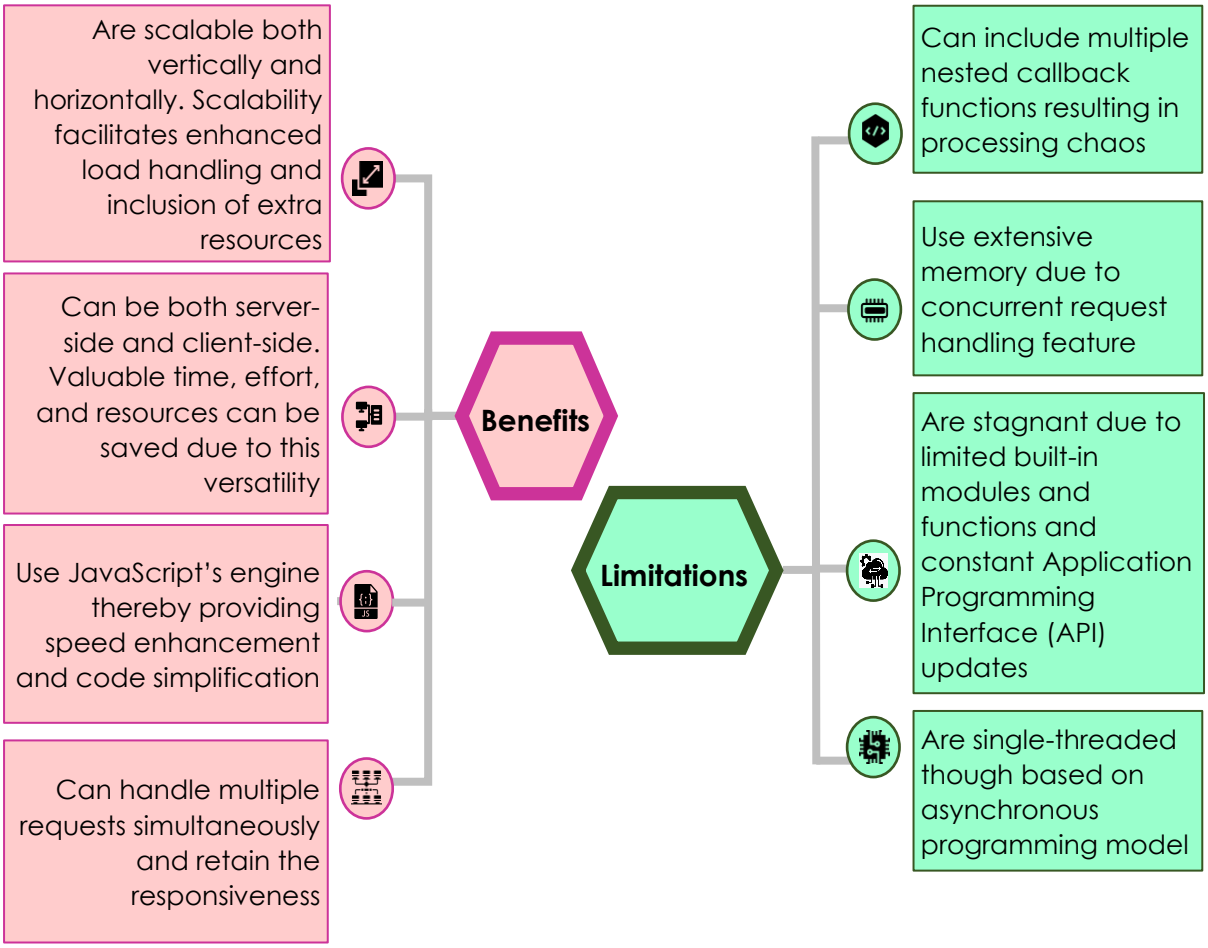


A set of specific instructions to the server is known as a thread in Node.js.

2.2 Benefits and Limitations of Node.js

Software developers who are well-versed with JavaScript will find it quite easy to comprehend Node.js in their applications. They can develop programs using JavaScript for both front-end and back-end applications. However, Node.js comes with its own set of limitations as well.

Benefits and limitations of Node.js applications are presented here.
Node.js applications:



2.3 Console-based Application in Node.js

There are two ways to develop Node.js applications, namely console-based and Web-based. Console-based Node.js applications run at the command prompt. These applications do not have a Graphical User Interface (GUI).

The `console` module in Node.js has the `console.log` function that helps to display custom messages on the VS Code terminal or command prompt.

Consider an example code where a custom message using `console.log` function will be displayed on the terminal.

Perform following steps:

1. Open a text file in VS Code.
2. Type the given code in the file.

```
console.log('This session covers the essentials of
Node.js');
```

In the example code, a custom message is provided as the argument for `console.log` function.

3. Save the file as `sample_display.js` in a folder on the local system.
4. Open the command prompt window and navigate to the required folder. Alternatively, open VS Code Terminal. Ensure that the environment path variable includes the path for the Node.js installation.
5. To run the saved program, at the command prompt or VS Code Terminal, type following command and press Enter.

```
node sample_display.js
```

Figure 2.2 displays the output of the code.

```
This session covers the essentials of Node.js
```

Figure 2.2: Console Output

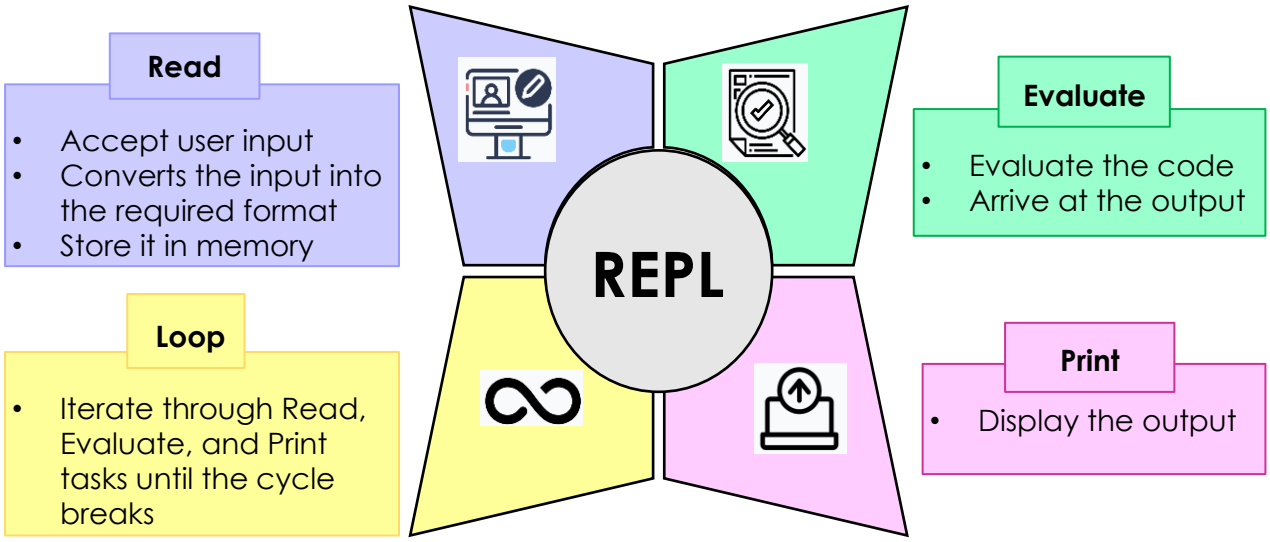
Observe the program output. The custom message is displayed on the command prompt or Terminal.

2.3.1 Overview of REPL

In Node.js, REPL stands for Read Evaluate Print Loop. REPL resembles a Windows command-line environment or a Unix/Linux shell, installed with Node.js on the system.

The conventional way of executing a Node.js program is to run the saved `.js` file at the command prompt. However, using REPL, small code snippets can be directly executed at the REPL prompt without creating `.js` files.

Each of the four parts in REPL plays a specific role. Tasks performed by these four parts are as follows:



To activate the REPL prompt in Windows, perform following steps:

1. Open the command prompt window.
2. At the command prompt, type the given command and press Enter.

```
node
```

Figure 2.3 displays the command output at the command prompt.

```
Welcome to Node.js v21.1.0.
Type ".help" for more information.
>
```

Figure 2.3: Command Output to Launch REPL

3. The REPL command prompt appears.
4. Begin executing the Node.js commands.

Working with Simple Expressions

Simple mathematical expressions can be executed directly at the REPL command prompt.

Consider an example to understand this. Perform the given steps:

1. Activate the REPL prompt in Windows.
2. Type the mathematical expression at the REPL prompt and press Enter.

```
15+80*5
```

Figure 2.4 displays the output of the mathematical expression.

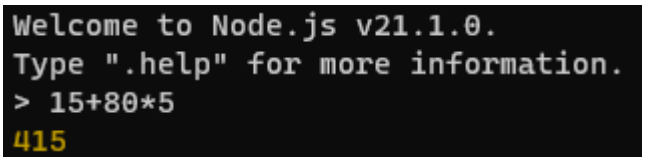


Figure 2.4: Output of Mathematical Expression

Observe the output. The result of the mathematical expression is displayed on the terminal. A separate `.js` file has not been created to store or process the mathematical expression.

Similar to simple mathematical expressions, string expressions can also be executed at the REPL command prompt.

Consider an example where a simple string expression using a concatenation operator is executed at the REPL command prompt.

Perform following steps:

1. Activate the REPL prompt in Windows.
2. Type the given string expression at the REPL prompt and press Enter.

```
'Welcome' + ' ' + 'to Node.js'
```

Figure 2.5 displays the output of the string expression.

Figure 2.5: Output of String Expression

Observe the output. The concatenation operator joins two strings and produces a single string. The result of the string expression is displayed on the terminal.

Working with Variables

A variable is a programming entity that provides a name to a specific location in system's memory. A value can be stored in a variable and that value can be

used in programs through the variable. In Node.js code, use the `var` keyword to declare and assign a value to the variable.

Consider an example where variables are used to demonstrate an addition operation.

Perform following steps:

1. Activate the REPL prompt in Windows.
2. Type the statements given in Code Snippet 1 at the REPL prompt one by one. Ensure to press Enter at the end of each statement for execution.

Code Snippet 1:

```
var a = 5
var b = 7
var c = a + b
console.log ('The sum of a and b is:',c);
```

In Code Snippet 1, values are stored in variables `a` and `b` respectively. Variable `c` stores the sum of variables `a` and `b`. The result of addition is displayed as the output.

Figure 2.6 displays the output of the addition operation.

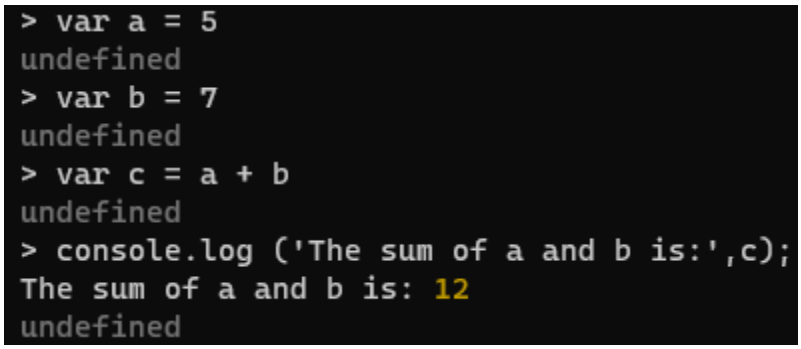


Figure 2.6: Output of Addition Using Variables

Observe the output, which is, The sum of 5 and 7 is 12.

Note that by default the return value of the commands is displayed as undefined after the execution of each command. This can be suppressed by setting the `ignoreUndefined` option to `true` when starting an REPL instance as given in following command:

```
node -e "require('repl').start({ignoreUndefined: true})"
```

The default value is `false`.

Working with Multiline Expressions

The REPL environment is able to identify complete expressions at the prompt. If the Enter key is pressed at the end of an incomplete expression, REPL automatically adds ellipsis (…) on the next line. The ellipsis is an indication to continue the code.

Consider an example that demonstrates the use of REPL multiline expressions with the help of a `do-while` loop.

Perform following steps:

1. Activate the REPL prompt in Windows.
2. Type the given statements in Code Snippet 2 at the REPL prompt one by one. Ensure to press Enter at the end of each statement for continuity.

Code Snippet 2:

```
var a = 5
var b = 7
var i = 0
do{
     var c = a + b
     i++
     a++
     b++
     console.log ('Sum of a and b in
     iteration',i,'is', c)
}while (i<3)
```

In Code Snippet 2, variables `a`, `b`, and `i` are declared and assigned values. Inside the `do-while` loop:

- Variables `a` and `b` are added.
- The result is stored in variable `c`.
- The values of `a`, `b`, and `i` are incremented by 1.
- The result of addition obtained in each iteration is displayed on the terminal.

The `do-while` loop continues to execute until the value of `i` is less than 3.

Figure 2.7 displays the output of the multiline expressions.

```
> var a = 5
undefined
> var b = 7
undefined
> var i = 0
undefined
> do{
... var c = a + b
... i++
... a++
... b++
... console.log ('Sum of a and b in iteration', i, 'is', c)
... }while (i<3)
Sum of a and b in iteration 1 is 12
Sum of a and b in iteration 2 is 14
Sum of a and b in iteration 3 is 16
undefined
```

Figure 2.7: Output of Multiline Expressions

Observe the output. Ellipsis are automatically generated to continue the code. Results of the iterations are displayed as 12, 14, and 16.

Table 2.1 lists some of the commonly used commands in Node.js REPL.

Commands	Description
Ctrl + C/Ctrl + c	Stops the current command's execution
Ctrl + C twice/Ctrl + c twice/Ctrl + d	Exits the REPL environment
up/down keys	Helps to check the command history and update the earlier commands
tab keys	Displays a list of current commands
`.help`	Shows a list of all available commands
`.break/.clear`	Exits from multiline expressions
`.save <filename>`	Saves the current REPL session to a file provided in the command
`.load <filename>`	Loads the specified file contents into the current REPL session

Table 2.1: REPL Commands

2.4 Node.js Programming Constructs

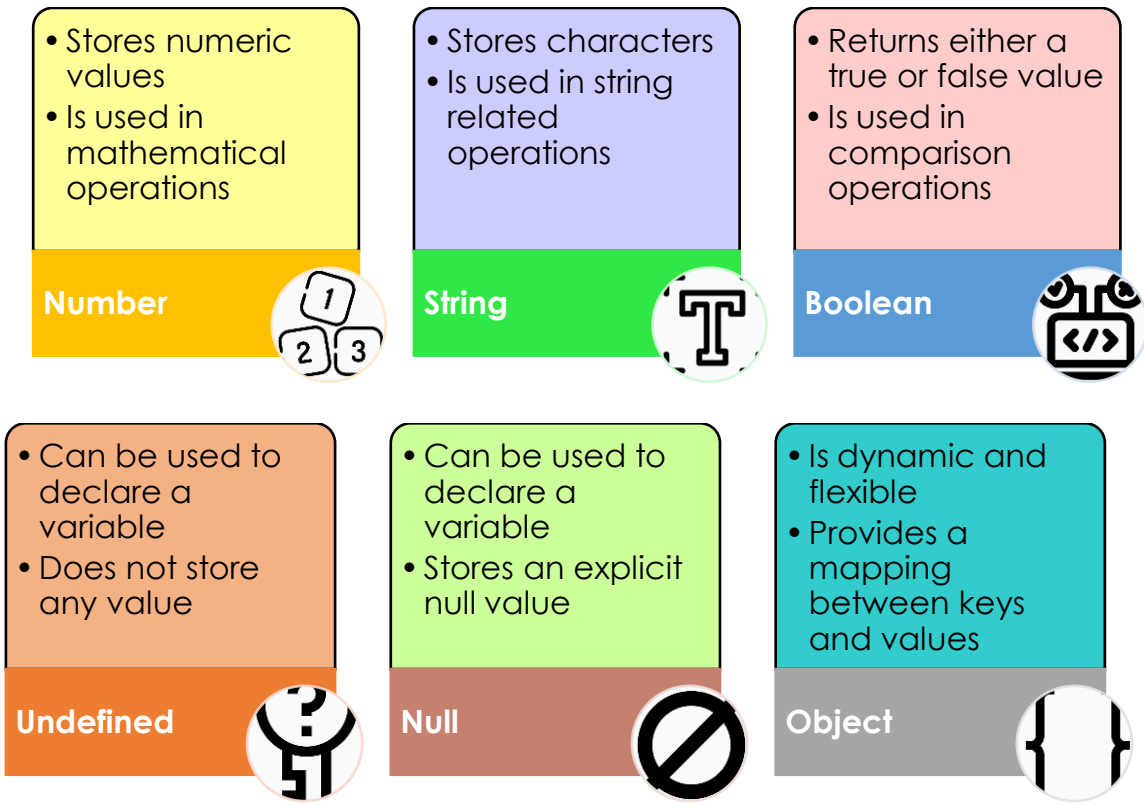
Different types of programming constructs are available in Node.js. These constructs enable the developers to create varied forms of applications.

Some of the programming constructs in Node.js are as follows:

2.4.1 Overview of Data Types

Data types in a programming language are important entities. The type of value stored in a variable signifies the importance of a data type. Variables can store different types of values such as text, numbers, decimals, and so on. For each variable, a data type can be assigned.

Here are various data types that can be used in Node.js applications.



Working with Number Data Type

The `number` data type can take both positive and negative values. Consider an example code in Code Snippet 3 where variables are declared using number data types and arithmetic operations are demonstrated.

Code Snippet 3:

```
var a = 40;
var b = 10.5;
```

```
var c = -4;
console.log('Value of a is:',a);
console.log('Value of b is:',b);
console.log('Value of c is:',c);
console.log('Sum of a and c is :' , (a+c));
console.log('Subtracting b from c is:' , (c - b));
console.log('Product of c and b is :' , (c * b));
console.log('Quotient of a/b is :' , (a / b));
console.log('Remainder of a/b is :' , (a % b));
```

In Code Snippet 3, although `var` is used, variables `a`, `b`, and `c` are inferred to be of `number` data type based on the content they hold. Various mathematical operations such as addition, subtraction, multiplication, and division are carried out in the program.

Figure 2.8 displays the output of Code Snippet 3.

```
Value of a is: 40
Value of b is: 10.5
Value of c is: -4
Sum of a and c is : 36
Subtracting b from c is: -14.5
Product of c and b is : -42
Quotient of a/b is : 3.8095238095238093
Remainder of a/b is : 8.5
```

Figure 2.8: Output of Code Snippet 3

Observe the program output. The results of the mathematical operations are displayed on the console.

Working with String Data Type

To assign a value to a string data type variable, use either single quotes or double quotes. The variable becomes a string variable only after the value in the required format is assigned to it.

Consider an example code in Code Snippet 4 where string concatenation is demonstrated.

Code Snippet 4:

```
var str1 = 'Welcome ';
var str2 = 'to Node.js Session 2';
console.log('First string is:',str1);
console.log('Second string is:',str2);
console.log('Concatenated string is:',str1+str2);
```

In Code Snippet 4, two string variables are declared. These two strings are concatenated using the + concatenation operator.

Figure 2.9 displays the output of Code Snippet 4.

```
First string is: Welcome
Second string is: to Node.js Session 2
Concatenated string is: Welcome to Node.js Session 2
```

Figure 2.9: Output of Code Snippet 4

Observe the program output. The first string, second string, and the concatenated string are displayed.

Working with Boolean Data Type

The Boolean data type helps the developers to build comparison and conditional logic in Node.js applications.

Consider an example code in Code Snippet 5 where Boolean data type variables are demonstrated.

Code Snippet 5:

```
var i = false;
var a = 6;
var b = 9;
console.log('The Boolean value of variable i is:', i);
console.log('The value of variable a is:',a);
console.log('The value of variable b is:',b);
console.log('The values of variables a and b are the
same:',a == b);
console.log('The value of variable a is not the same
as value of variable b:',a!= b);
console.log('The Boolean value of !i is:', !i);
```

In Code Snippet 5, variable `i` is declared using a Boolean data type. Operators such as `equal to`, `not equal to`, and `not` are used to compare the values.

Figure 2.10 displays the output of Code Snippet 5.

```
The Boolean value of variable i is: false
The value of variable a is: 6
The value of variable b is: 9
The values of variables a and b are the same: false
The value of variable a is not the same as value of variable b: true
The Boolean value of !i is: true
```

Figure 2.10: Output of Code Snippet 5

Observe the program output. The Boolean values are displayed as `true` or `false`.

Working with Undefined Data Type

Consider an example code in Code Snippet 6 where undefined data type variables are demonstrated.

Code Snippet 6:

```
var number1;
console.log('Value of number1 is:',number1);
var number2;
console.log('Value of number2 is:',number2);
```

In Code Snippet 6, two variables are declared without any values.

Figure 2.11 displays the output of Code Snippet 6.

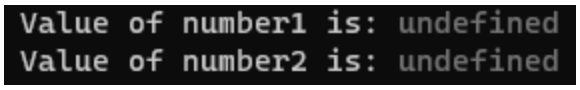


Figure 2.11: Output of Code Snippet 6

Observe the program output. Since there are no values assigned to the variables, the output shows `undefined`.

Working with Null Data Type

Consider an example code in Code Snippet 7 where null data type variables are demonstrated.

Code Snippet 7:

```
var number1 = null;
console.log("Value of number1 is:",number1);
var number2 = null;
console.log("Value of number2 is:",number2);
```

In Code Snippet 7, two variables are declared and assigned null values.

Figure 2.12 displays the output of Code Snippet 7.

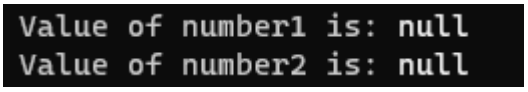
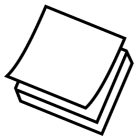


Figure 2.12: Output of Code Snippet 7

Observe the program output. The values are displayed as `null`.

Working with Object Data Type

Names and values play a crucial role in an object data type. Each variable declared as an object will have name:value pairs. Names must be unique. Each name will have a corresponding value. However, values can be repeated. Commas separate the name:value pairs and they are presented in curly braces. Another term for name:value pairs is object properties.



Name:value pairs for the object data type are also known as key: value pairs in Node.js.

Consider an example code in Code Snippet 8 where object data type is demonstrated.

Code Snippet 8:

```
var student = {
 Name: 'David Franklin',
 Age: 12,
 Grade: 8
};
console.log('Name:',student.Name);
console.log('Age:',student.Age);
console.log('Grade:',student.Grade);
```

In Code Snippet 8, the variable `student` is declared as an object data type with the key:value pairs. Keys are `Name`, `Age`, and `Grade`. Values are `David Franklin`, `12`, and `8`.

Figure 2.13 displays the output of Code Snippet 8.

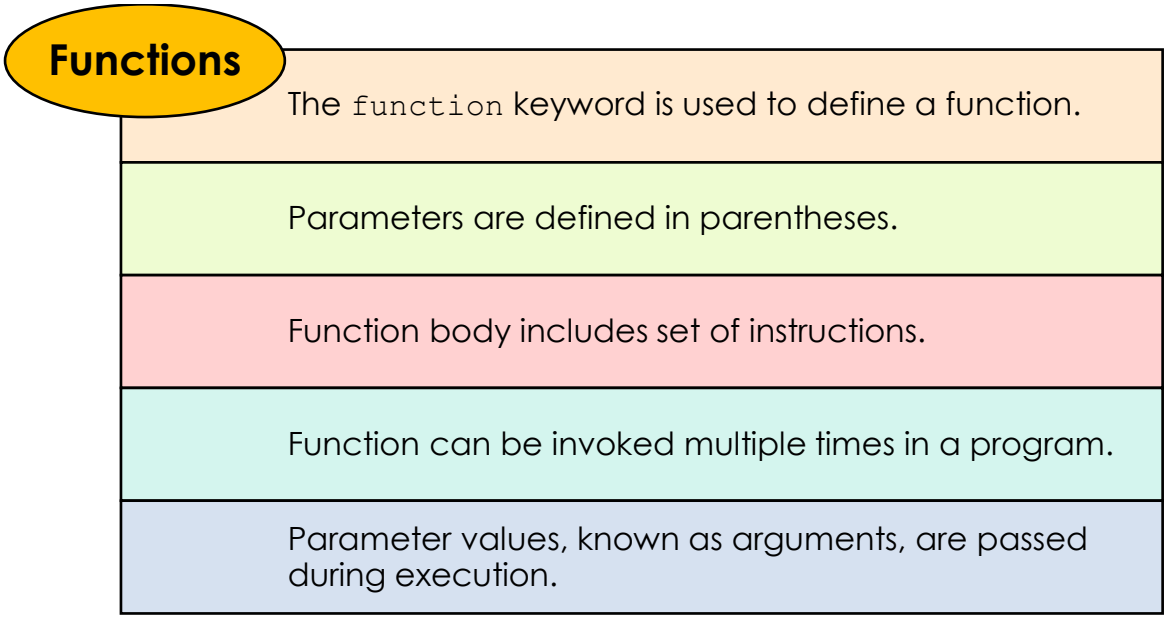
```
Name: David Franklin
Age: 12
Grade: 8
```

Figure 2.13: Output of Code Snippet 8

Observe the program output. The keys and the values of the object are displayed.

2.4.2 Overview of Functions

Functions are an important programming construct in Node.js. There are two types of functions, built-in and user-defined.



Consider an example code in Code Snippet 9 where a user-defined function is created and executed.

Code Snippet 9:

```
function student(name,age,grade,mark1, mark2) {
     console.log('Name:',name);
     console.log('Age:',age);
     console.log('Grade:',grade);
     var total_marks=mark1+mark2;
     console.log('Total Marks:',total_marks)
}
student('David Franklin',13,8,78,85);
```

In Code Snippet 9, a user-defined function with the name `student` is defined. The function calculates total marks of a student and has five parameters. Values for these parameters are passed as arguments when the function is called.

Figure 2.14 displays the output of Code Snippet 9.

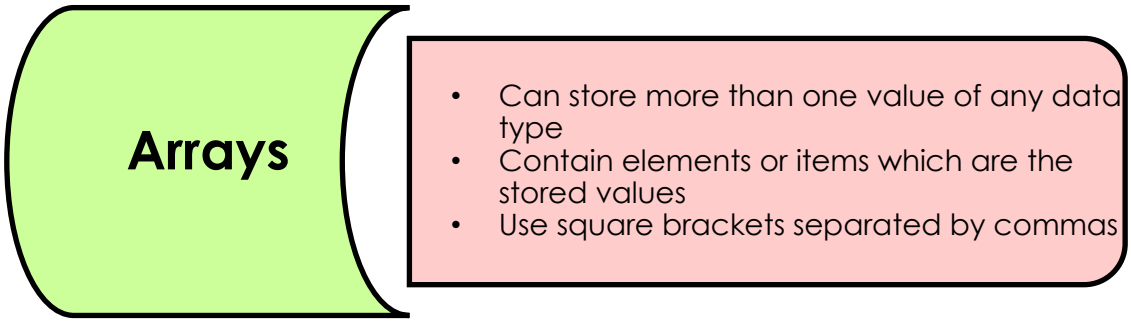
```
Name: David Franklin
Age: 13
Grade: 8
Total Marks: 163
```

Figure 2.14: Output of Code Snippet 9

Observe the program output. The name, age, grade, and marks are passed as arguments when the function is called. Total marks is calculated and the values of the object are displayed.

2.4.3 Overview of Arrays

Arrays are another frequently used programming construct in Node.js.



Consider an example code in Code Snippet 10 where an array is declared and demonstrated.

Code Snippet 10:

```
var arr1 = [76,80,85,90,92];
var sum=0, avg=0;
for (var i = 0; i < arr1.length; i++) {
     sum=sum+arr1[i]
     avg=sum/5
}
console.log('Average marks of five subjects:',avg);
```

In Code Snippet 10, an array is declared, which stores the marks of five subjects obtained by a student. The total marks and average marks obtained by a student are calculated.
Figure 2.15 displays the output of Code Snippet 10.

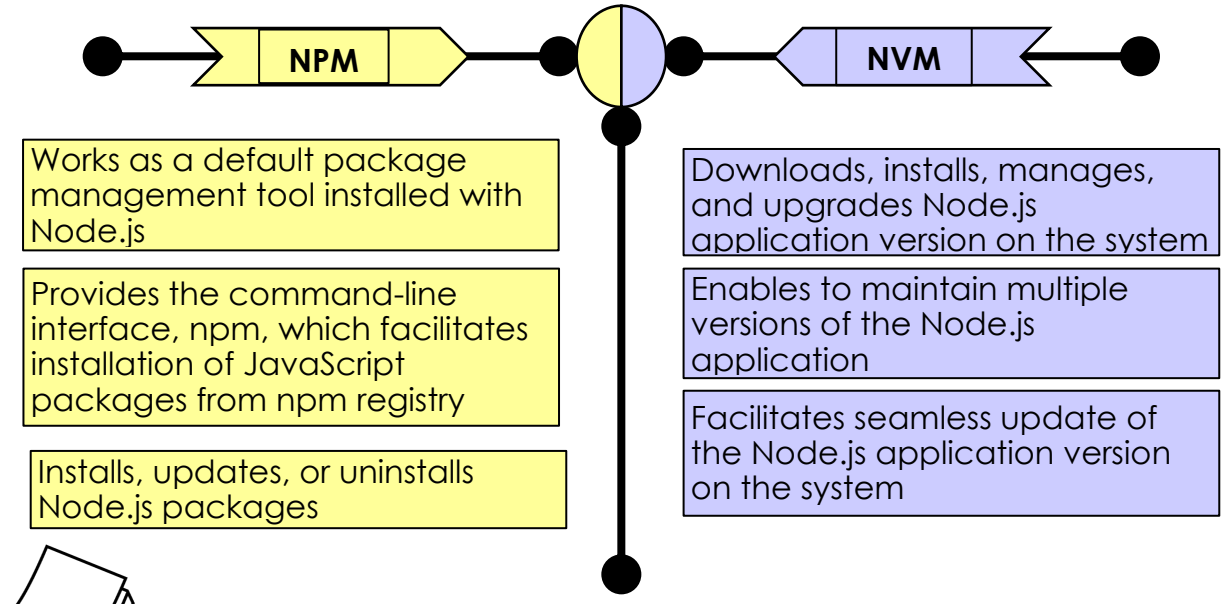
```
Average marks of five subjects: 84.6
```

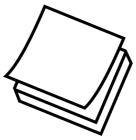
Figure 2.15: Output of Code Snippet 10

Observe the program output. The average marks calculated is displayed.

2.5 NPM and NVM

Node Package Manager (NPM) and Node Version Manager (NVM) are important tools for managing Node.js and JavaScript development.



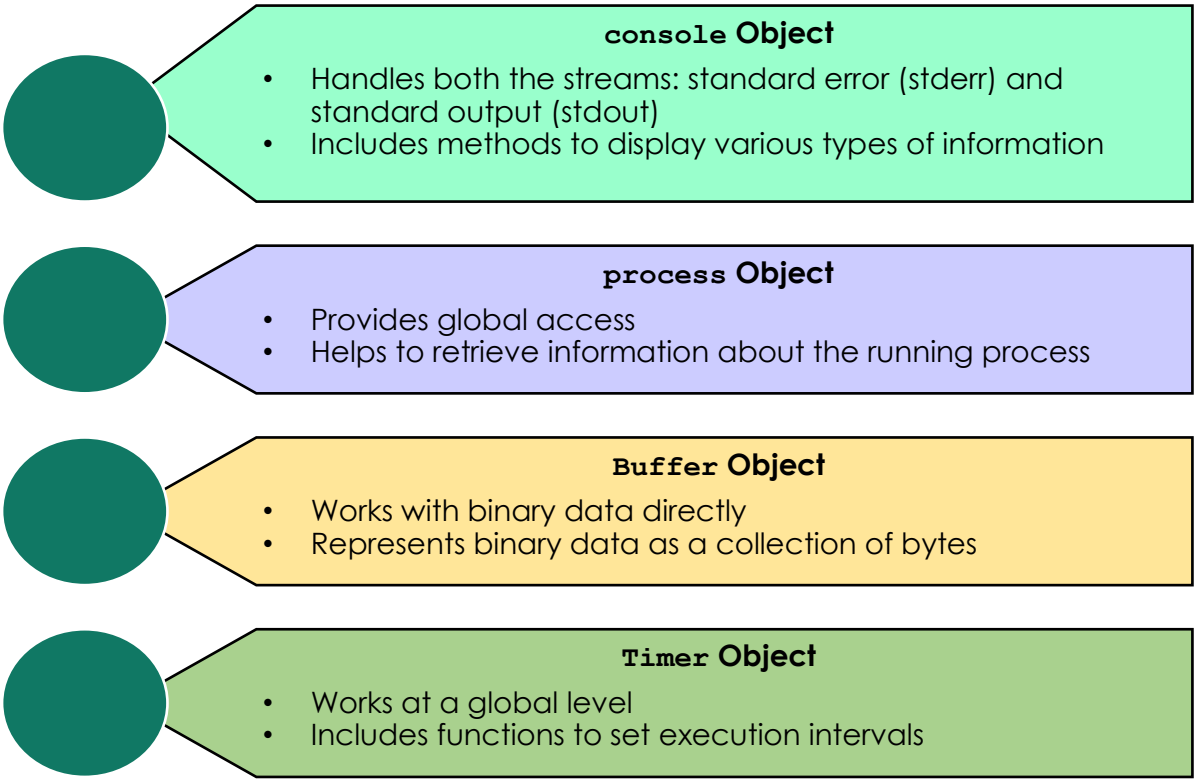


The `npm` registry, an online repository, contains both private and public packages.

2.6 Global Objects

There are certain objects in Node.js that are global in nature. Such objects can be used directly in Node.js applications. There is no compulsion to import any module before using the global objects.

Some of the commonly used global objects in Node.js are as follows:



Working with Global Objects

Here are some code snippets on the global objects.

`console` object

Consider an example code in Code Snippet 11 where the `console` object is demonstrated.

Code Snippet 11:

```
const console = require('console');

console.log('console.log is used to display a simple
message');

number = 100
if (number > 150) {
 console.log('Number is less than 100');
}
else {
 console.error('Error message: Number is greater than 100');
}

console.error('console.error is used to display an error
message');
```

```
a = 110;
b = 0;
a = a/b;

function divbyzero(a, b) {

 if (a == 0) {
  console.warn(`Result is ${b}`);
 }
 else {
  console.warn(`Divide by zero is not possible`);
}
}
divbyzero(a, b);

console.warn('console.warn is used to display a warning
message');

console.info('Node.js follows the concurrent mode of
executing the tasks');

console.info('console.info is used to display an
informational message');
```

In Code Snippet 11, various methods from the `console` class are included. Each method has its own functionality.

Figure 2.16 displays the output of Code Snippet 11.

```
console.log is used to display a simple message
Error message: Number is greater than 100
console.error is used to display an error message
Divide by zero is not possible
console.warn is used to display a warning message
Node.js follows the concurrent mode of executing the tasks
console.info is used to display an informational message
```

Figure 2.16: Output of Code Snippet 11

Observe the program output. The messages are displayed on the console.

`process` object

Consider an example code in Code Snippet 12 where the `process` object is demonstrated.

Code Snippet 12:

```
console.log(`Current directory: ${process.cwd()}`);
console.log(`Command-line arguments: ${process.argv}`);
```

In Code Snippet 12, the `process.cwd` method is used to access the current working directory of the program. The `process` object `argv` is used to print the command-line arguments supplied to the Node.js program using the `process.argv` property.

Figure 2.17 displays the output of Code Snippet 12.

```
C:\Node Js>node process_global.js
Current directory: C:\Node Js
Command-line arguments: C:\Program Files\nodejs\node.exe,C:\Node Js\process_global.js
```

Figure 2.17: Output of Code Snippet 12

Observe the program output. The current working directory and the command-line arguments are displayed.

`Buffer` object

Consider an example code in Code Snippet 13 where the `Buffer` object is demonstrated.

Code Snippet 13:

```
var buf_obj = Buffer.from('Learning');
console.log(buf_obj);
```

In Code Snippet 13, the string '`Learning`' is converted into binary data.

Figure 2.18 displays the output of Code Snippet 13.

```
<Buffer 4c 65 61 72 6e 69 6e 67>
```

Figure 2.18: Output of Code Snippet 13

Observe the program output. The binary form of the string data is displayed.

Timer object

Consider an example code in Code Snippet 14 where the Timer object is demonstrated.

Code Snippet 14:

```
function display() {
 console.log('Welcome to Session 2 of Node.js');
}
const timeinterval = setInterval(display, 1000);
setTimeout(() => {clearInterval(timeinterval);}, 3000);
```

In Code Snippet 14, the `display` method is executed each second. The `setInterval` method executes the `display` method continuously in milliseconds, according to the intervals specified.

After three seconds, the `clearInterval` method is called with the interval ID given by the `setInterval` method. The `clearInterval` method terminates the continued execution of the `display` method.

Figure 2.19 displays the output of Code Snippet 14.

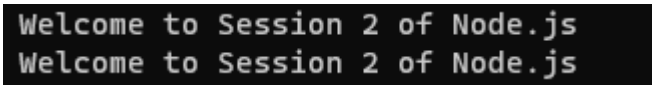


Figure 2.19: Output of Code Snippet 14

Observe the program output. The messages are displayed only twice as the `display` function is terminated.

2.7 Summary

- Node.js, a JavaScript-based runtime environment, has various components to handle the client request, complete the execution, and provide the response.
- Node.js manages multiple client requests simultaneously.
- Node.js is single-threaded and follows an asynchronous programming model.
- REPL environment reads the input, evaluates the code, prints the output, and iterates through the tasks.
- Console-based applications perform their tasks in command-line environments.
- NVM manages the versions of Node.js application.
- NPM installs, updates, and uninstalls Node.js packages and modules.
- Global objects in Node.js are available throughout the lifetime of an application.

Test Your Knowledge

1. What is the primary function of the event loop in Node.js?

    a) Storing client requests
    b) Processing client requests with blocking I/O operations
    c) Handling external resources
    d) Sending responses to clients

2. Which of the following are benefits of building server-side applications in Node.js?

    a) Easy scalability
    b) Concurrent Request Handling
    c) Single-Threaded Limitation
    d) Memory Usage

3. Which method will you use to display the given text on console?

    `Node.js is an efficient runtime environment based on JavaScript.` on the console.

    a) `console.display()`
    b) `console.write()`
    c) `console.log()`
    d) `console.message()`

4. Which of the following environment will you use to quickly test some JavaScript code?

    a) Node.js IDE
    b) Web browser console
    c) REPL
    d) Command prompt

5. Which keyword is used to declare variables in JavaScript?

    a) `let`
    b) `const`
    c) `def`
    d) `var`

Answers to Test Your Knowledge

1	d
2	a, b
3	c
4	c
5	d

Try It Yourself

1. Write a program to perform arithmetic operations on the given set of numbers. Initialize three variables `x`, `y`, and `z` with the values `27`, `7.2`, and `-3`, respectively. Perform the given operations and display the results:

   a. Display the sum of `x` and `z`
   b. Display the result of subtracting `y` from `z`
   c. Display the product of `z` and `y`
   d. Display the quotient of `x` divided by `y`
   e. Display the remainder when `x` is divided by `y`

2. Write a program to perform operations on boolean values and numeric variables. Initialize a boolean variable `i` with the value `false` and two numeric variables `x` and `y` with the values `10` and `1`. Perform the given operations and display the results:

   a. Display if the value of `x` is the same as the value of `y`
   b. Check and display if the value of `x` is not same as the value of `y`
   c. Display the boolean value of the negation of variable `i`

3. Write a program to perform string concatenation. Initialize two string variables, `text1` with the value `Welcome to` and `text2` with the value `JavaScript Programming`. Perform the given operations and display the results:

   a. Display the value of the `text1` variable
   b. Display the value of the `text2` variable
   c. Concatenate the two strings `text1` and `text2` and display the result

4. Write a program to display information about `cricketers`. Define a JavaScript object named `cricket` with properties such as `Name`, `Run`, and `Wickets`. Perform the given operations and display the results:

   a. Display the `Name` of the cricketer
   b. Display the `Run` of the cricketer
   c. Display the number of `Wickets` taken by team

5. Create a function named `salesPersonInMarketing` that takes the given parameters:

   `name`: The name of the salesperson.
   `age`: The age of the salesperson.
   `experience`: The years of experience in sales.
   `salesInQuarter1`: The sales achieved in the first quarter.
   `salesInQuarter2`: The sales achieved in the second quarter.

   The function should perform the given operations and display the results:

   a. Display the `name` of the salesperson
   b. Display the `age` of the salesperson
   c. Display the years of `experience` in sales
   d. Calculate and display the total sales achieved in both quarters

6. Write a program to calculate the average test scores of a cricketer across five test matches performed in the World Cup. You have an array named `testScoresMatch` that has the test scores of the cricketer. Calculate the average test score and display the result.

   a. Create an array `testScoresMatch` with the test scores of a cricketer using this data [`111, 55, 303, 324, 400`]
   b. Calculate the total `sum` of test scores
   c. Calculate and display the `average` test score